

GeoCadastra compared with proxy_project

Reviewed the current working tree on 14 September 2026. This is a source-code review of the local copies, not an allegation about the author or a claim about an unseen deployed service. The reviewed proxy_project folder was subsequently removed at your request; its source-path references below are historical and no longer open locally. Several files in your own app and frontend were already uncommitted; those changes were preserved.

Main finding: GeoCadastra has implemented geometry, persistence, model and worker code, with an operator frontend now present. The reviewed BhoomiDrishti interface contains real React/Leaflet screens and some real Turf geometry operations, but its principal training/inference demonstration produces canned logs, predetermined scores and invented locations. Those screens do not establish a functioning cadastral ML pipeline. GeoCadastra still has substantial gaps before a real-data deployment; its synthetic tests and runnable training code do not establish real-world boundary accuracy.

The clearest unsupported claims in proxy_project

What the interface presents	What the reviewed implementation does	Consequence
Five training epochs with improving loss and accuracy	The loss/accuracy lines are literal strings in <code>logMessages</code> ; a timer releases them while advancing progress by 0.8 every 180 ms.	These are animation messages, not observations from an optimizer or GPU.
An inference result with 98.4% confidence and IoU 0.887	<code>handleComplete</code> writes literal values into <code>sessionStorage</code> .	These numbers are not calculated by comparing predictions with held-out labels.
Hundreds of newly extracted parcels	The count is the character-code sum of the selected patch name modulo 400, plus 900.	Renaming a patch can change the alleged count without changing its pixels.
Predicted location on the map	Latitude/longitude are randomly sampled inside a state's bounding rectangle, with a random central-India fallback.	This is not image georeferencing, and the rectangle does not even guarantee the point is inside the state.
DSM and DTM processing	The displayed layers are existing image/mask/binary PNGs. The log claiming an average 4.2 m DSM-minus-DTM difference is a string.	A segmentation mask is not an elevation raster. No measured height follows from this path.
Parcel confidence and ownership inventory	<code>generateMicroParcels</code> invents rectangles, identifiers, owners/status and confidence between 85 and 99.9.	The inventory can be a demonstration dataset, but cannot establish property records or calibrated model confidence.

Evidence: **training animation and layer definitions** [proxy_project/BhoomiDrishti_AI/artifacts/bhoomi-ai/src/App.tsx:890], **completion handler** [proxy_project/BhoomiDrishti_AI/artifacts/bhoomi-ai/src/App.tsx:1200], **parcel generator** [proxy_project/BhoomiDrishti_AI/artifacts/bhoomi-ai/src/Utils/parcelGenerator.ts:35].

These findings are directly reproducible from the code: the confidence and IoU are constants; parcel count does not consume image data; the training log contains every epoch result before any computation starts. I did not execute a GPU training process in that project because no such process is called by this interface path. A file search found a shapefile-conversion Python script, but did not find a Python training notebook or model-weight file in the inspected tree. The code-path evidence is stronger than relying on file extensions alone.

Topology: there is geometry code, but the success report overstates it

The frontend loads parcel-like features, or falls back to a square grid. It then translates, rotates and scales these shapes around the selected city. It creates anomalies from templates and seeded arithmetic. This is not a scan of measured city parcel geometry simply because a real city name appears above the map.

Its `autoFix` attempts a Turf union of the first two features. That is a real polygon operation. It does not demonstrate repairing every listed overlap, sliver, ring, encroachment or disconnection. The exception is swallowed, and the code then reports `fixed: errors.length` and clears all errors regardless of whether the union succeeded. Merging neighbouring property polygons can also destroy their separate identity; removing an overlap is not by itself a valid cadastral correction.

The Express `/topology/fix` path separately assigns zero errors and `fixedCount: 12`. It does not call a geometry validator in that handler.

Evidence: **geometry generation** [proxy_project/BhoomiDrishti_AI/artifacts/bhoomi-ai/src/components/TopologyModule.tsx:294], **auto-fix** [proxy_project/BhoomiDrishti_AI/artifacts/bhoomi-ai/src/components/TopologyModule.tsx:527], **API topology handlers** [proxy_project/BhoomiDrishti_AI/artifacts/api-server/src/routes/bhoomi.ts:341].

Persistence and exports

The reviewed proxy routes use module-level arrays and objects. Updating a parcel mutates the array; starting a processing run inserts a record with fixed progress and descriptive step names. These handlers do not establish durable database storage or a dispatched training job. A database package in `package.json` does not change what those handlers execute. Its export handler returns a filename and `ready` status; the handler does not generate or return the named archive. This finding concerns that API endpoint, not every separate browser-side export implementation.

Evidence: **in-memory parcels** [proxy_project/BhoomiDrishti_AI/artifacts/api-server/src/routes/bhoomi.ts:52], **processing and export routes** [proxy_project/BhoomiDrishti_AI/artifacts/api-server/src/routes/bhoomi.ts:284].

Stage-by-stage comparison

Stage	GeoCadastra implementation	proxy_project implementation reviewed	What remains for GeoCadastra
0. Input data	Reproducible synthetic wards; real GeoTIFF/parcel ingestion helper; durable raster files. Separately, actual LINZ imagery and elevation have now been downloaded.	Static image/mask assets, geographic reference assets and browser selection flows.	The main HTTP intake and new frontend still create synthetic wards. Downloaded data is not automatically ingested into the production pipeline.
1. Parcel representation	Nodes, shared edges and faces represent adjacency.	Independent polygon features are rendered or generated.	Real reference-map alignment and appropriate CRS support must be established for a New Zealand end-to-end run.
2. Durable edits	Postgres/PostGIS changesets, geometry checks, versions and provenance.	Reviewed API mutates in-memory records; UI stores results in sessionStorage.	Operational deployment, migrations and broker failure testing need their own verification.
3. Parcel assignment	Evidence-weighted assignment and recorded-area refinement are implemented.	Parcels are generated/scattered; inference count is derived from patch name.	Capacity still solves selected exact targets within tolerances, rather than directly solving all allowed area intervals; the previously identified pinned-boundary case remains a concern.
4. ML training/inference	Actual PyTorch model, optimizer, losses, resumable checkpoints and tiled inference. Worker uses model evidence when weights are configured.	Timer-driven training display and constant result metrics in the traced path.	Close measured synthetic targets, prepare real supervised training and evaluate on unseen real locations. Buying a GPU alone guarantees none of these.
5. Evidence fusion	Weighted node estimates, disagreement handling and validated moves.	Mask/binary display and static height claims in the traced inference screen.	Demonstrate the contribution of real model evidence and real survey observations on a matched dataset.
6. Uncertainty calibration	Calibration code exists; API currently reports <code>not_calibrated</code> .	Confidence figures are constants or generated numbers.	Connect calibration to held-out model errors; do not label raw model variance or sigmoid output as certified confidence.

Stage	GeoCadastra implementation	proxy_project implementation reviewed	What remains for GeoCadastra
7. Survey planning	Standalone planning logic exists.	Visible verification queues and parcel-status controls.	Expose and validate the planner through the application; a queue is not evidence of an uncertainty-driven survey plan.
8. Application	FastAPI, Celery worker code and an actual frontend calling those endpoints.	More extensive React/Leaflet demonstration screens and Express routes, with the limitations above.	Complete real-data upload, deployment and browser-level workflow checks.

GeoCadastra sources: **synthetic generator** [geocadastra/synth/generator.py], **real intake** [geocadastra/jobs/ingest.py:30], **changesets** [geocadastra/store/changeset.py], **capacity refinement** [geocadastra/core/capacity.py:135], **model** [geocadastra/models/backbone.py:96], **training** [geocadastra/models/train.py:61], **worker model selection** [geocadastra/jobs/orchestrator.py:212], **uncalibrated API status** [geocadastra/api/main.py:561], **frontend API client** [frontend/src/api.ts:8].

Corrections to the earlier comparison

Your project now has a frontend. Saying its entire disadvantage is that it has no visible interface is outdated. I inspected the current API client and its call sites, and freshly ran `npm run build` in `frontend`: TypeScript checking and the Vite production build passed. This is not a browser interaction test or proof that a live worker/database are configured correctly.

Your worker also now has a model-evidence path. Saying the pipeline can never use the model is outdated. However, simulation remains the default when `GEOCADAstra_MODEL_WEIGHTS` is unset. The model path loads the expected default MultiTaskNet and uses stored rasters when available. This distinction must be visible in any demonstration claim.

The new low-cost New Zealand Colab model is a separate RGB boundary baseline. Its checkpoint is not compatible with this worker. The new `GeoCadastra_Stage4_GPU.ipynb` instead trains the actual MultiTaskNet on synthetic wards and produces the worker's expected checkpoint structure. Format compatibility is not evidence of real-world accuracy.

Your remaining weaknesses, without giving either project a free pass

1. The main intake still expects the application's fixed CRS and casts imagery to uint8. The New Zealand data is EPSG:2193; blindly putting it into an India-specific coordinate system is not a sound solution. A production import needs a suitable coordinate-system design and robust nodata/dtype handling.

2. The model loader is cached by filename and loads on CPU. Replacing a checkpoint at the same path can leave a long-running worker using old weights. Use immutable checkpoint filenames immediately; fix cache identity before relying on hot updates.
3. The area-refinement implementation still selects exact feasible targets and uses equality constraints. This review confirms that structure remains. I did not rerun the earlier adversarial pinned-boundary reproduction in this turn; its prior finding should not be represented as freshly tested here.
4. Existing acceptance-test source still marks the boundary-error/uncertainty target as expected failure. I did not rerun that long test today. Neither its comments nor a GPU purchase establish that compute is the only remaining cause.
5. LINZ parcel maps, survey_area attributes, imagery and elevation have distinct provenance and capture dates. Their availability is valuable, but the full reconstruction experiment still needs independent inputs, a clean evaluation split and survey-quality checks.
6. The 100-image PNG/height export supplies real height values, not 100 newly prepared supervised parcel-training examples. Calling it ready for the existing Colab training loader would be inaccurate until the labels and manifest are prepared.

What to demonstrate next

First run the actual-model GPU notebook's one-epoch benchmark, then a bounded resumable training run and held-out evaluation. Keep the checkpoint and measured results together. In parallel, inspect the matched New Zealand image/height data, prepare parcel labels and spatial splits, and compare real-image reconstruction with and without model evidence using identical input records. Show actual failures and `not_calibrated` where applicable.

A fair demo claim today is: “GeoCadastra implements constrained parcel geometry, persistent edits and a configurable model-evidence path; we are measuring model accuracy and integrating real datasets.” The reviewed proxy inference screen supports the narrower claim: “This interface demonstrates the intended workflow using stored assets and simulated outputs.” It does not support the displayed measured-training or cadastral-accuracy claims.

Additional finding from the training smoke test: the existing Stage 4 visible-boundary distance is one-sided. A nearly untrained model predicted every pixel as boundary, which gives zero distance from reference boundaries to predictions even though the prediction is useless. The uncertainty target failed in this smoke run, so the combined result remained false. The GPU wrapper now also reports predicted boundary fraction, prediction-to-reference distance and precision within one pixel, and refuses an all-image boundary prediction. This demonstrates why an impressive single distance score is insufficient; the underlying acceptance test should also be strengthened with false-positive-sensitive metrics.

Final Colab review and fixes

The latest training implementation batches wards and uses disjoint validation seeds. A new regression found and fixed a resume defect: latest checkpoints were saved before updating the best validation loss, so a later worse epoch could replace the best file after restart. The latest state now records the updated minimum, and resume derives the minimum from validation history, including recovery of older affected checkpoints.

The final notebook mounts Drive before training, retains Colab's matching CUDA PyTorch installation, benchmarks one epoch, resumes the same 400-epoch schedule and evaluates the selected checkpoint directly. The pytest acceptance command in the pasted instructions creates a separate training run and is not an evaluation command for saved weights. Batch size one can still execute on GPU; it does not literally imply CPU execution speed.

Fresh validation for this update: 15 training/resume/NZ-checkpoint tests passed; the frontend TypeScript/Vite production build passed. These checks do not constitute a full database/broker acceptance run or GPU accuracy measurement. The most valuable next experiment is a measured run of the actual network, followed by a spatially held-out real-data comparison of model evidence versus the simulated baseline. Fix the remaining interval-feasibility issue before claiming all valid area constraints are handled.